SESIÓN 4 · SEMANA 4 · UNIDAD 3

Control de Versiones y Automatización DevOps

Git, branching, versionamiento semántico y pipelines CI/CD

Checkpoint del proyecto: Actividad 3 (30%)

Agenda de hoy (120 minutos)

10 min	Bienvenida y objetivos de la sesión
15 min	Checkpoint: Actividad 3 - qué se evalúa
50 min	Git, branching, SemVer, CI/CD y cultura DevOps
20 min	Demo en vivo: pipeline mínimo funcionando
15 min	Taller: definir la estrategia de branching del equipo
10 min	Cierre y próximos pasos

Punto de control del proyecto

Implementación de control de versiones y automatización DevOps

30%

de la nota final

1.25	Uso correcto, estructurado y profesional de Git
1.0	Estrategia de branching y versionamiento clara y bien aplicada
1.25	Pipeline CI/CD funcional, completo y automatizado
0.75	Trazabilidad y control de cambios verificable
0.75	Documentación y evidencias claras y completas

CONTROL DE VERSIONES

Git: control distribuido como cambio de paradigma

1. Cada desarrollador opera con una copia completa del repositorio, incluyendo toda la historia, sin depender de un servidor central.

2. Más allá de lo técnico, Git funciona como mecanismo de trazabilidad: cada commit registra qué cambió, quién y cuándo.

3. En proyectos bien gestionados esa historia es documentación técnica; en proyectos mal gestionados, existe pero no sirve para nada.

90%+

de los desarrolladores profesionales usan Git hoy

Creado por Linus Torvalds en 2005 para el kernel de Linux.

El poder y el riesgo de las ramas

1. Las ramas independientes permiten que varias funcionalidades coexistan sin interferirse.

2. Las ramas longevas se distancian progresivamente de la línea principal y su integración se vuelve costosa.

3. No existe una estrategia universalmente correcta: GitFlow, trunk-based development y monorepos resuelven problemas distintos.

El error más frecuente

Adoptar una estrategia de branching porque es popular, no porque resuelva un problema concreto del equipo.

Versionamiento semántico (SemVer)

Un contrato de comunicación: MAYOR . MENOR . PARCHE

MAYOR

Cambios incompatibles

Rompen la compatibilidad con versiones anteriores.

MENOR

Funcionalidad nueva

Compatible con versiones anteriores.

PARCHE

Corrección de errores

No altera la interfaz externa del sistema.

?

¿Por qué importa?

Permite evaluar el impacto de una actualización sin inspeccionar el código fuente.

AUTOMATIZACIÓN

Si duele, hazlo más frecuentemente

1. La integración continua parte de una premisa contraintuitiva: si integrar código causa conflictos, la solución es integrar más frecuente, no menos.

2. La CI solo es efectiva si se combina con pruebas automatizadas que detecten regresiones reales, no un teatro de la automatización.

3. El despliegue continuo introduce riesgos si se confunde con un pipeline de semáforo verde permanente que da una falsa sensación de seguridad.

130,000+

despliegues diarios en empresas con prácticas maduras de CI/CD

Ejemplo de referencia de la industria.

Las tres vías

1. Flujo: hacer visible y optimizar el movimiento del trabajo de un extremo a otro.
2. Retroalimentación: detectar problemas en la fuente, en ambas direcciones.
3. Aprendizaje continuo: tratar cada incidente como información, no como fracaso.

Lo que las herramientas no pueden hacer

Los fracasos en la adopción de DevOps rara vez son tecnológicos: ocurren cuando se instalan pipelines sin transformar los incentivos del equipo. (Kim et al., 2016)

Taller: estrategia de branching del equipo (15 min)

1 **Elijan GitFlow, trunk-based o GitHub Flow**

Justificando la elección para el tamaño de su equipo.

2 **Definan la convención de nombres de ramas**

Y el proceso de revisión (pull/merge request).

3 **Redacten BRANCHING.md**

El archivo que documenta la estrategia elegida.

4 **Identifiquen la primera etapa del pipeline**

¿Qué se automatiza primero: build, test o análisis estático?

Para reflexionar y debatir

R

Reflexionar: ¿qué estrategia de branching sería más adecuada para el sistema del proyecto? ¿Cómo cambia esa decisión si el equipo crece de 2 a 10 personas?

D

Debatir: ¿CI/CD sin una cultura de calidad real es más peligroso que no tenerlo, porque da una falsa sensación de seguridad? ¿Bajo qué condiciones la automatización se convierte en un riesgo?

Lecturas para la Sesión 5 (Unidad 4)

OBLIGATORIAS

Fowler, M., Beck, K., Brant, J., Opdyke, W., & Roberts, D. (1999). Refactoring (cap. 1-2). Addison-Wesley.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. (2017). Site Reliability Engineering (1.a ed., cap. 1-3). Helion.

ISO/IEC. (2018). Software Maintenance Standard (ISO/IEC 14764).

COMPLEMENTARIAS

Prometheus. (2024). Documentation. prometheus.io/docs/

Grafana Labs. (2024). Grafana Documentation. grafana.com/docs/

Antes de la Sesión 5

- Tener el pipeline funcionando: build, pruebas, análisis estático y despliegue a un ambiente de pruebas
- Documentar la estrategia de branching en BRANCHING.md
- Leer los materiales de la Unidad 4 (diapositiva anterior)
- Empezar a identificar qué mejora de mantenimiento propondrán en la Unidad 4

Nos vemos en la Sesión 5 · Unidad 4 · Mantenimiento y evolución